

Tối ưu hóa hàm mất mát Ridge

cho bài toán định giá điện thoại đã qua sử dụng

Nguyễn Hồng Phúc (25007860)
Phạm Huy Hoàng (25007878)
Bùi Quang Huy (25007877)

Trường Đại học Khoa học Tự nhiên - ĐHQGHN
Khoa Toán - Cơ - Tin học
Môn: Tối ưu hóa nâng cao

Ngày 13 tháng 8 năm 2026

Tối ưu hóa hàm mất mát Ridge

cho bài toán định giá điện thoại đã qua sử dụng

Nguyễn Hồng Phúc (25007890)
Phạm Huy Hoàng (25007878)
Bùi Quang Huy (25007877)

Trường Đại học Khoa học Tự nhiên - ĐHQGHN
Khoa Toán - Cơ - Tin học
Môn Tối ưu hóa ràng buộc

Ngày 11 tháng 8 năm 2025

0:00–0:40. Chào thầy cô và các bạn. Nhóm em trình bày bài tập môn Tối ưu hóa nâng cao, nội dung là so sánh các thuật toán bậc một và bậc hai trên hàm mất mát Ridge.

Bài toán định giá điện thoại đã qua sử dụng chỉ là chỗ để có một hàm mục tiêu thật và đủ lớn, còn thứ nhóm em đo là cách đi tìm w chứ không phải chất lượng dự đoán giá. Mong thầy cô đọc các biểu đồ theo hướng đó.

Đặt vấn đề

Khảo sát tham số

So sánh tổng hợp

So sánh với thư viện

Kết luận



[Dội vấn đề](#)[Khảo sát tham số](#)[So sánh tổng hợp](#)[So sánh với thư viện](#)[Kết luận](#)

0:40–1:15. Bài nói đi theo bốn phần. Phần đầu dựng hàm mục tiêu và ba hằng số L , μ , κ , vì mọi kết luận về sau đều quy về ba con số đó. Phần hai khảo sát tham số của từng thuật toán, phần ba đặt cả bảy

thuật toán lên cùng một biểu đồ, phần cuối đối chiếu với scikit-learn.

Sau slide kết luận còn mười một slide phụ lục. Nhóm em để dành cho phần hỏi nên sẽ không chiếu, trừ khi thầy cô hỏi tới.

- ▶ Dữ liệu định giá điện thoại đã qua sử dụng (Kaggle): 1 000 000 bản ghi, 28 cột, không giá trị thiếu và không bản ghi trùng lặp, nên phần chuẩn bị chỉ còn mã hóa và chuẩn hóa.
- ▶ Thí nghiệm chạy trên mẫu 200 000 bản ghi, $n = 160\,000$ điểm huấn luyện, vì một lần tính gradient trên toàn bộ dữ liệu mất 271 ms thay vì 39 ms.
- ▶ $d = 280$ sau mã hóa, gồm 21 cột định lượng gốc, 210 tích từng cặp, 14 bình phương và 35 cột one-hot: biến tương tác sinh ra phần lớn kích thước bài toán.
- ▶ Biến mục tiêu lấy logarit vì giá bán lại lệch phải rõ rệt, trung vị 18 555 so với trung bình 22 598.
- ▶ Trọng tâm của bài tập là phần tối ưu hóa chứ không phải phần mô hình hóa.



- ▶ Dữ liệu định giá điện thoại di động (Kaggle): 1.000.000 bản ghi, 28 cột, không giá trị thiếu và không trùng lặp, cần phân chuẩn bị chỉ còn mã hóa và chuẩn hóa.
- ▶ Thí nghiệm chạy trên mẫu 200.000 bản ghi, $n = 100.000$ điểm huấn luyện, vì một lần tính gradient trên toàn bộ dữ liệu mất 271 ms thay vì 39 ms.
- ▶ $d = 280$ sau mã hóa, gồm 21 cột định lượng gốc, 210 tính tổng cặp, 14 tính phương và 35 cột one-hot biến tương tác sinh ra phần lớn kích thước bài toán.
- ▶ Biến mục tiêu lấy logarit vì giá bán lại lệch phải rõ rệt, trung vị 18.555 so với trung bình 22.598.
- ▶ Trọng tâm của bài tập là phần tối ưu hóa chứ không phải phần mô hình hóa.

1:15–2:45. Dữ liệu lấy từ Kaggle, một triệu bản ghi và 28 cột, không thiếu giá trị và không trùng lặp, nên phần chuẩn bị chỉ còn mã hóa với chuẩn hóa.

Nhóm em chạy trên mẫu 200.000 dòng chứ không chạy cả triệu, và động cơ ở đây là chi phí tính toán chứ không phải cỡ mẫu thống kê: trên toàn bộ dữ liệu, một lần tính gradient mất 271 ms thay vì 39. Chênh bẩy lần tự nó nhỏ, nhưng cả lưới tham số có hàng chục nghìn vòng lặp nên nó nhân lên thành hàng chục giờ.

Sau mã hóa thì $d = 280$, trong đó 210 cột là tích từng cặp, tức phần lớn kích thước bài toán do biến tương tác sinh ra chứ không do dữ liệu gốc. Biến

mục tiêu lấy logarit vì giá bán lại lệch phải mạnh, trung vị 18.555 so với trung bình 22.598, nên mọi RMSE trong báo cáo đọc trên thang logarit và không quy ra tiền được.

Câu cần nhấn ở slide này: hàm mục tiêu đóng băng trước khi chạy thí nghiệm. Nếu giữa chừng có ai sửa dữ liệu thì mọi biểu đồ trước đó mất giá trị so sánh, do các đường không còn chung một đích.

Nếu bị hỏi lấy mẫu có làm hỏng kết quả không: không, với các kết luận về tốc độ hội tụ, vì chúng chỉ phụ thuộc L , μ , κ . Nhưng có ảnh hưởng nếu đại lượng cần đo phụ thuộc thẳng vào n , chẳng hạn phương sai gradient của SGD.

Hàm mục tiêu và các hằng số quyết định tốc độ hội tụ

$$\min_{w \in \mathbb{R}^d} f(w) = \frac{1}{2n} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2, \quad L = \lambda_{\max}\left(\frac{1}{n}X^\top X\right) + \lambda, \quad \mu = \lambda_{\min}\left(\frac{1}{n}X^\top X\right) + \lambda, \quad \kappa = L/\mu$$

L	2,683
μ	0,01
κ	268,3
λ	0,01
f^*	0,023 743 0

Hessian $\frac{1}{n}X^\top X + \lambda I$ không phụ thuộc w , nên bài toán lồi mạnh và có nghiệm đóng w^* . Nhờ vậy f^* tính được trước khi chạy thuật toán nào, và mọi biểu đồ vẽ được sai số tuyệt đối $f(w_k) - f^*$ trên thang logarit.

Mã trận Gram có hạng 273 trên 280 và trị riêng kế tiếp chỉ cỡ $2 \cdot 10^{-8}$, tức phần đóng góp của dữ liệu vào μ coi như bằng không, nên $\mu = \lambda$ và $\kappa = L/\lambda$: hệ số hiệu chỉnh là đại lượng duy nhất quyết định số điều kiện.



$$\min_{w \in \mathbb{R}^n} f(w) = \frac{1}{2n} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2, \quad L = \lambda_{\max}\left(\frac{1}{n}X^TX\right) + \lambda, \quad \mu = \lambda_{\min}\left(\frac{1}{n}X^TX\right) + \lambda, \quad \kappa = L/\mu$$

L	2.683	Hessian $\frac{1}{n}X^TX + \lambda I$ không phụ thuộc w , nên bài toán
μ	0.01	là thuần và có nghiệm đóng w^* . Nhờ vậy f^* tính được
κ	268.3	trước khi chạy thuật toán nào, và mọi biểu đồ vẽ được
λ	0.01	sai số tuyệt đối $f(w_k) - f^*$ trên thang logarith.
f^*	0.023 743 0	Ma trận Gram có hạng 273 trên 280 và trị riêng sát sát

chỉ cỡ $2 \cdot 10^{-8}$, nên phần đóng góp của dữ liệu vào μ coi như bằng không, nên $\mu \approx \lambda$ và $\kappa \approx L/\lambda$: hệ số hiệu chỉnh chính là đại lượng duy nhất quyết định số đầu kiện.

2:45–4:30. Về đầu của f là trung bình bình phương sai số, về sau phạt trọng số lớn. Chia cho n để L không phụ thuộc số mẫu, nhờ vậy đặt cạnh SGD mới công bằng, còn chia thêm cho 2 là để số 2 triệt tiêu khi lấy đạo hàm.

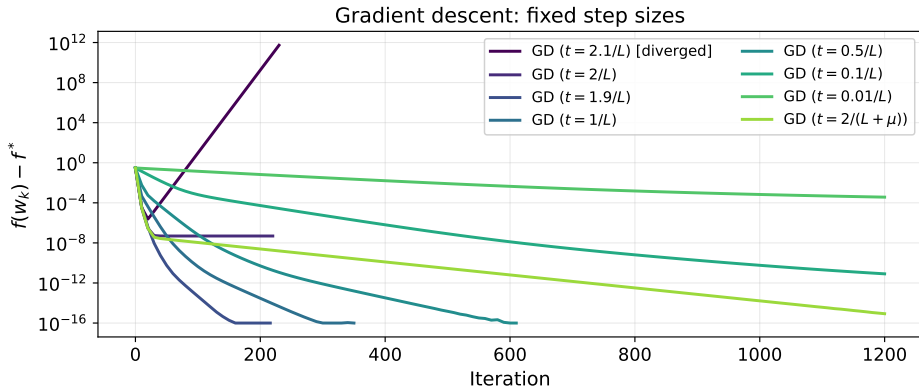
Hessian không phụ thuộc w , tức chỗ nào cũng cong như nhau, nên mặt hàm mục tiêu là một cái bát parabol và bài toán có nghiệm đóng. Nhờ đó nhóm em biết trước $f^* = 0,023\,743\,0$, và mọi biểu đồ vẽ được sai số tuyệt đối $f(w_k) - f^*$ trên thang logarith. Không có f^* thì chỉ so được với giá trị tốt nhất từng thấy, và đường nào cũng cắm xuống ở cuối biểu đồ, che mất hành vi thật.

Ba con số trong bảng: L là độ cong lớn nhất, μ là độ cong nhỏ nhất, κ là độ méo của cái bát. Bát méo 268 lần nghĩa là nó bị bóp thành một cái máng dài, hòn bi lăn ngang thì nhanh mà bò dọc theo máng thì chậm hơn nhiều bậc, và toàn bộ khó khăn của nhóm bậc một nằm ở đó.

Chỗ phải nói được là $\mu = \lambda$. Ma trận Gram có hạng 273 trên 280, trị riêng kế tiếp chỉ cỡ $2 \cdot 10^{-8}$, nhỏ hơn L tám bậc, nên phần đóng góp của dữ liệu vào μ coi như bằng không. Vậy $\kappa = L/\lambda$: hệ số hiệu chỉnh quyết định số điều kiện, dữ liệu không có tiếng nói.

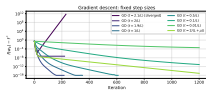
Nếu bị hỏi khi nào kết luận đó đảo chiều: khi mọi cột độc lập tuyến tính và trị riêng nhỏ nhất cùng bậc với λ , khi ấy μ quay về do dữ liệu quyết định.

Gradient descent: độ dài bước cố định



Chênh 10% quanh ngưỡng $2/L = 0,745$ đã đổi hẳn kết cục: $t = 2.1/L$ đẩy hàm mục tiêu lên $5,4 \times 10^{11}$, còn $t = 1.9/L$ hội tụ sau 20 vòng lặp.





Chính 10% quanh ngưỡng $2/L \approx 0,745$ đôi đôi hẳn lẫn các: $t = 2,1/L$ đẩy hàm mục tiêu lên $5,4 \times 10^{11}$, còn $t = 1,9/L$ hội tụ sau 20 vòng lặp.

4:30–6:00. Gradient descent chỉ có một tham số là độ dài bước t . Bước ngắn quá thì lâu mới tới đáy, bước dài quá thì vọt qua đáy sang sườn bên kia, có khi lên cao hơn cả chỗ vừa đứng.

Mời thầy cô nhìn đường đi lên ở phía trên biểu đồ. Đó là $t = 2.1/L$, nhóm em cố ý cho chạy vượt ngưỡng, và nó đẩy hàm mục tiêu lên $5,4 \times 10^{11}$. Ngay cạnh đó, $t = 1.9/L$ hội tụ sau 20 vòng lặp. Hai cấu hình chênh nhau 10% quanh $2/L = 0,745$ mà kết cục lật hẳn.

Cơ chế nằm ở từng hướng riêng. Sau mỗi vòng, sai số theo hướng ứng với trị riêng λ_i nhân với $|1 - t\lambda_i|$,

nên muốn tắt dần thì hệ số ấy phải nhỏ hơn 1. Hướng dốc nhất có $\lambda_i = L$, do đó khi t vượt $2/L$ thì $|1 - tL| > 1$ và thành phần đó nhân lên mỗi vòng thay vì tắt đi. Đúng tại ngưỡng, hệ số bằng 1, tức thành phần đó đứng im mãi, và vì vậy $t = 2/L$ kẹt ở $4,9 \times 10^{-8}$ chứ không đi tiếp được.

Trong vùng an toàn thì chia đôi t làm số vòng lặp nhân đôi, thấy rõ ở dãy 20, 40, 80 ứng với $1.9/L$, $1/L$, $0.5/L$. Kết luận thực dụng: chọn t càng sát $2/L$ càng tốt.

Nếu bị hỏi về $t = 2/(L + \mu)$: đó là giá trị tối ưu về lý thuyết, nhưng ở đây nó chỉ hơn $1.9/L$ có 0,07 giây, vì $\mu \ll L$ nên $2/(L + \mu) \approx 2/L$.

Gradient descent: backtracking và cách chọn độ dài bước

$\alpha = 0.5$	Đánh giá hàm / vòng lặp	Vòng lặp đạt 10^{-6}	Thời gian (s)
$\beta = 0.5$	1,39	15	0,852
$\beta = 0.8$	2,37	20	1,617
$\beta = 0.9$	3,96	20	1,799

- ▶ β quyết định chi phí, còn α gần như không ảnh hưởng, vì ba giá trị $\alpha \in \{0.1, 0.3, 0.5\}$ chỉ chênh nhau 4,8% về số lần đánh giá hàm.
- ▶ Trong vùng ổn định, số vòng lặp tỉ lệ nghịch với độ dài bước, 20, 40, 80 vòng ứng với $t = 1.9/L$, $1/L$, $0.5/L$, nên chọn t càng gần $2/L$ càng tốt.
- ▶ Backtracking chậm hơn bước cố định tốt nhất 10% (0,852s so với 0,773s), và khoản chênh đó mua lấy chỗ không phải biết L .



$\alpha = 0.5$	Đánh giá hàm / vòng lặp	Vòng lặp đạt 10^{-6}	Thời gian (s)
$\beta = 0.5$	1,39	15	0,032
$\beta = 0.8$	2,37	20	1,617
$\beta = 0.9$	3,96	30	1,799

- β quyết định chi phí, còn α gần như không ảnh hưởng, vì ba giá trị $\alpha \in \{0.1, 0.3, 0.5\}$ chỉ chênh nhau 4,8% và số lần đánh giá hàm.
- Trong vòng ổn định, số vòng lặp tỉ lệ nghịch với độ dài bước: 20, 40, 80 vòng ứng với $\tau = 1.9/L$, $1/L$, $0.5/L$, nên chọn τ càng gần $2/L$ càng tốt.
- Backtracking chậm hơn bước cố định tức nhất 10% (0,032 s so với 0,773 s), và khoản chênh đó mua lấy chỗ không phải biết L .

6:00–7:45. Bước cố định đòi biết L trước. Không tính được L thì thử: bắt đầu bằng một bước dài, thấy chưa giảm đủ thì nhân với $\beta < 1$, lặp tới khi điều kiện Armijo chấp nhận. Nói nôm na, điều kiện ấy bắt bước hiện tại phải giảm được ít nhất α phần của mức giảm mà độ dốc hứa hẹn.

Cột giữa của bảng là số lần đánh giá hàm mục tiêu trên mỗi vòng lặp, đo trong 100 vòng đầu. Đọc dọc thì β quyết định chi phí: $\beta = 0.5$ tốn 1,39 lần, $\beta = 0.9$ tốn 3,96 lần, vì β càng gần 1 thì mỗi lần rút ngắn càng ít nên phải thử nhiều lần hơn. Đọc ngang thì đổi α gần như không đổi gì, chênh nhiều nhất 4,8%.

Cặp β này còn là ví dụ sạch nhất trong cả bài cho việc vì sao đề bài bắt vẽ hai biểu đồ. $\beta = 0.8$ và

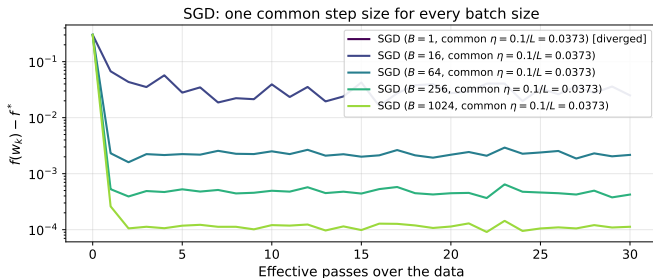
$\beta = 0.9$ gần trùng nhau trên trục vòng lặp nhưng tách hẳn trên trục thời gian, vì chênh lệch nằm ở số lần đánh giá hàm bên trong mỗi vòng, mà trục vòng lặp không nhìn thấy chi phí đó.

Chốt lại, backtracking chậm hơn bước cố định tốt nhất 10%, và khoản chênh đó mua lấy chỗ không phải biết L . Bài này L tính dễ nên bước cố định hợp lý hơn, nhưng nhận xét ấy đảo chiều ngay khi d lớn tới mức không tính nổi trị riêng lớn nhất của ma trận Gram.

Nếu bị hỏi vì sao chỉ lấy 100 vòng đầu: phép đo phải nằm trong giai đoạn thuật toán còn tiến triển. Khi f đã chạm giới hạn số học, Armijo không thỏa mãn được với bất kỳ bước nào và con số bị thổi lên gấp năm lần.

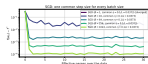
SGD: độ dài bước theo kích thước lô

$$L = 2,683, \quad L_{\max} = \max_i \|x_i\|^2 + \lambda = 3109,7 \quad \Rightarrow \quad L_B = L + \frac{L_{\max} - L}{B}, \quad \eta_0 = 1/L_B$$



Dùng chung $\eta = 0.1/L$ thì lô kích thước 1 phân kỳ thành NaN ngay trong lượt duyệt (epoch) đầu tiên, còn lô 1024 đạt sai số $1,1 \times 10^{-4}$. Ngưỡng ổn định của một bước ngẫu nhiên phụ thuộc độ cong của các mẫu trong lô chứ không phụ thuộc L , mà hai hằng số này chênh nhau 1159 lần, nên chỉ lô lớn mới nằm trong vùng ổn định của một bước tính theo L .

$$L \approx 2,683, \quad L_{\max} \approx \eta \mu_0 \|w\|^2 + \lambda \approx 3109,7 \quad \Rightarrow \quad L_B \approx L + \frac{L_{\max} - L}{B}, \quad \eta = 1/L$$



Dùng chung $\eta = 0.1/L$ thì lô kích thước 1 phân kỳ thành NaN ngay trong lượt duyệt đầu tiên, còn lô 1024 đạt sai số $1,1 \times 10^{-4}$. Ngưỡng ổn định của một bước ngẫu nhiên phụ thuộc độ cong của các mẫu trong lô chứ không phụ thuộc L , mà hai hằng số này chênh nhau 1159 lần, nên chỉ lô lớn mới nằm trong vùng ổn định của một bước tính theo L .

7:45–9:15. Mỗi vòng gradient descent phải quét cả 160 000 dòng để tính một gradient. SGD chỉ lấy ngẫu nhiên B dòng rồi bước luôn, nên hướng đi lệch lạc mà số bước đi được trong cùng thời gian nhiều hơn hẳn.

Cái bẫy nằm ở hai hằng số trên đầu slide. L của cả hàm là 2,683, còn $L_{\max}$, tức L của mẫu đơn lẻ dốc nhất, đo được 3109,7, lớn hơn 1159 lần. Nghĩa là có những dòng dữ liệu mà riêng nó tạo ra một cái bát dốc đứng gấp nghìn lần cái bát trung bình.

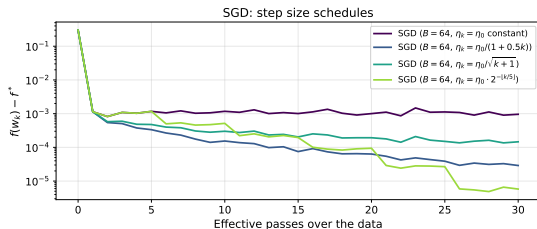
Nhóm em làm thí nghiệm đối chứng để chỉ ra chỗ đó: dùng chung $\eta = 0.1/L$ cho mọi kích thước lô thì $B = 1$ phân kỳ thành NaN ngay trong lượt duyệt đầu tiên, còn $B = 1024$ vẫn về được $1,1 \times 10^{-4}$. Ngưỡng

ổn định của một bước ngẫu nhiên phụ thuộc độ cong của các mẫu trong lô chứ không phụ thuộc L , nên chỉ lô lớn mới nằm trong vùng an toàn của một bước tính theo L .

Cách xử lý là cho mỗi kích thước lô một bước riêng, $\eta_0 = 1/L_B$. Công thức L_B nội suy giữa hai đầu: $B = 1$ thì $L_B \approx L_{\max}$, còn B lớn thì $L_B \rightarrow L$.

Nếu bị hỏi vì sao SGD không có biến thể backtracking: hai lý do. Một, quá đắt, vì Armijo cần đánh giá f trên toàn bộ dữ liệu ở mỗi lần thử bước, mà với $B = 64$ thì một lượt duyệt có 2500 lần cập nhật, ước lượng 98 giây cho một lượt duyệt so với 0,082 giây đo được. Hai, không bảo đảm gì, vì Armijo áp lên gradient của lô chỉ bảo đảm giảm cho hàm mất mát của lô đó.

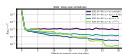
SGD: quy tắc chọn độ dài bước



Quy tắc	Sai số cuối
hằng	$9,58 \times 10^{-4}$
$\eta_0/\sqrt{k+1}$	$1,46 \times 10^{-4}$
$\eta_0/(1 + 0.5k)$	$2,90 \times 10^{-5}$
bậc thang	$5,80 \times 10^{-6}$

Bước hằng dừng ở một lân cận của w^* bán kính cỡ $\mathcal{O}(\eta\sigma^2/\mu)$, nên muốn sai số giảm tiếp thì phải hạ dần η_k để bán kính co theo.





Quy tắc	Sai số cuối
hằng	0.58×10^{-6}
$\eta_k = 1/k^2$	1.88×10^{-6}
$\eta_k(1 + 0.5k)$	2.90×10^{-6}
bậc thang	5.90×10^{-6}

Bước hằng dừng ở một lân cận của w^* bởi kính có $O(1/k^2)$, nếu muốn sai số giảm tiếp thì phải hạ dần η để bán kính co theo.

9:15–10:45. Đường nằm ngang trên biểu đồ là bước hằng, và nó không phải lỗi cài đặt. Gradient của một lô là gradient thật cộng nhiễu, mà càng tới gần đây thì gradient thật càng nhỏ trong khi nhiễu không nhỏ đi. Tới lúc nhiễu lấn át tín hiệu, thuật toán chỉ còn đi lung tung trong một lân cận quanh w^* , và trên thang logarit nó hiện ra thành đường ngang ấy. Nhóm em viết hẳn một phép thử tự động bắt buộc hành vi này phải xảy ra.

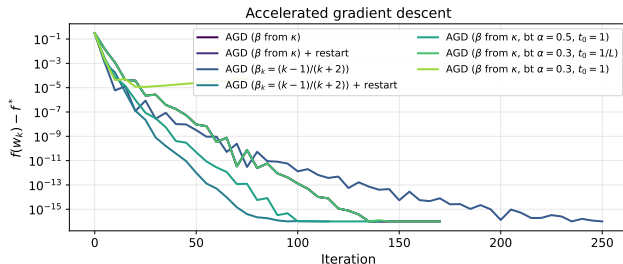
Bốn quy tắc trong bảng chạy cùng $B = 64$ và cùng 30 lượt duyệt, chỉ khác cách hạ η_k . Giảm càng nhanh thì sai số cuối càng nhỏ, vì bán kính lân cận co theo η_k , và quy tắc bậc thang hơn bước hằng 165 lần. Cả

bốn tốn thời gian như nhau, khoảng 2,4 giây, nên ở đây không có đánh đổi nào, bước hằng đơn thuần kém hơn.

Nói thêm một câu để chốt phần SGD: ngay cả quy tắc tốt nhất cũng chỉ xuống $5,8 \times 10^{-6}$ sau 30 lượt duyệt, tức chưa chạm ngưỡng 10^{-6} , trong khi GD cần đúng 20 vòng để đạt ngưỡng ấy. SGD trên bài này vì thế kém hơn các phương pháp còn lại, và nó chỉ đáng dùng khi một nghiệm tạm với chi phí thấp là đủ.

Nếu bị hỏi bốn quy tắc này có phải line search không: không, chúng ấn định η_k trước theo chỉ số vòng lặp và không đọc gì từ giá trị hàm mục tiêu.

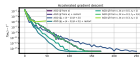
Accelerated gradient descent



Khởi động lại không đổi số vòng lặp đạt 10^{-6} , nhưng giảm tổng thời gian từ 10,5 s xuống 4,86 s vì nó khởi dao động ở phần đuôi.

- ▶ Với $\alpha = 0.3$ và $t_0 = 1$, phương pháp không hội tụ mà dừng ở $5,5 \times 10^{-5}$, vì line search chạy tại điểm ngoại suy y_k nên chỉ bảo đảm $f(w_{k+1}) < f(y_k)$ và Armijo chấp nhận t tới $1/\mu = 100$ trong khi ngưỡng ổn định là $2/L = 0,745$.
- ▶ Hai cách xử lý đều đưa phương pháp về hội tụ, lấy $\alpha = 0.5$ để điều kiện Armijo trùng bỏ đề giảm, hoặc khởi tạo $t_0 = 1/L$ để line search chỉ thu nhỏ từ một giá trị vốn an toàn. Momentum phải tính theo bước thực tế, $\beta = (1 - \sqrt{t\mu})/(1 + \sqrt{t\mu})$.





Khởi động tại không đổi và vòng lặp đạt 10^{-6} , nhưng giảm tổng thời gian từ 10,5 s xuống 4,86 s và số lần dao động ở phần đuôi.

- Với $\alpha = 0,3$ và $\beta_0 = 1$, phương pháp không hội tụ mà dừng ở $5,5 \times 10^{-5}$, vì line search chạy tại điểm ngoại suy y_k nên chỉ báo đảm $f(w_{k+1}) < f(y_k)$ và Armijo chấp nhận τ chỉ $1/\mu = 100$ trong khi ngưỡng ổn định là $2/(1 + \sqrt{\kappa}) = 0,745$.
- Hai cách xử lý đầu đưa phương pháp về hội tụ, lấy $\alpha = 0,5$ để điều kiện Armijo đúng bỏ để giảm, hoặc khởi tạo $\beta_0 = 1/L$ để line search chỉ thu nhỏ số một giá trị văn an toàn. Momentum phân tích theo bước thực tế, $\beta = (1 - \sqrt{\mu/L})/(1 + \sqrt{\mu/L})$.

10:45–12:15. Gradient descent như hòn bi lăn trong máng đầy mật, mỗi bước chỉ nghe theo độ dốc tại chỗ. AGD cho hòn bi có quán tính: nó nhớ hướng vừa đi và lao thêm theo hướng đó một chút. Nesterov khác momentum thường ở chỗ trượt tới điểm ngoại suy y_k trước rồi mới lấy gradient ở đó. Về lý thuyết, hệ số co rút cải thiện từ $1 - \mathcal{O}(1/\kappa)$ lên $1 - \mathcal{O}(1/\sqrt{\kappa})$, với $\kappa = 268$ thì nhanh hơn cỡ 16 lần ở phần đuôi.

Một kết quả ngược lý thuyết đáng nói: công thức $\beta = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ vốn tối ưu cho hàm lồi mạnh lại cần 35 vòng, còn công thức tăng dần $\beta_k = (k - 1)/(k + 2)$ chỉ cần 20. Vì công thức hằng nhắm vào trường hợp xấu nhất trong lớp hàm có κ cho trước, tức phổ trị riêng trải đều trên $[\mu, L]$,

mà bài toán này có phổ dồn cụm nên bộ tham số phòng thủ ấy mất lợi thế.

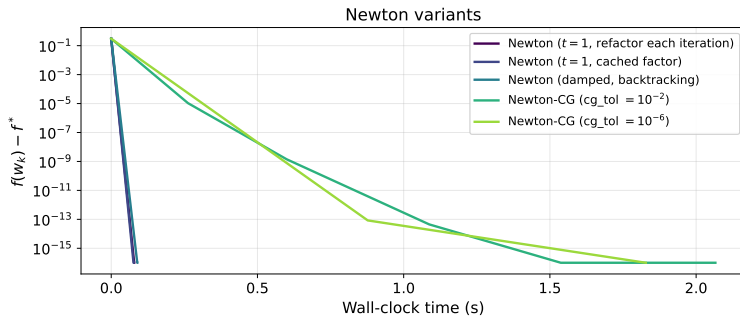
Momentum tăng dần gây dao động tuần hoàn, nhìn thấy như gợn sóng ở đuôi biểu đồ. Khởi động lại là khi phát hiện $\nabla f(y_k)^\top (w_{k+1} - w_k) > 0$, tức đang đi ngược hướng dốc, thì xả hết quán tính và tích lại từ đầu. Nó không đổi số vòng lặp đạt 10^{-6} nhưng giảm tổng thời gian từ 10,5 s xuống 4,86 s.

Nên kể chỗ này: lần cài đặt đầu, nhóm em để line search chọn t nhưng vẫn tính momentum theo $1/L$, và hàm mục tiêu bùng lên $1,47 \times 10^4$. Chương trình chạy trót lọt, không báo lỗi, chỉ trả về một con số trông hợp lý, nên chỉ có đối chiếu với mốc giải tích mới bắt được loại lỗi này.

Phương pháp Newton

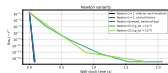
$$w_1 = w_0 - (\nabla^2 f)^{-1} \nabla f(w_0) = w^*$$

Với hàm bậc hai, một vòng lặp là đủ và bước Newton chính là nghiệm đúng, nên toàn bộ lời giải mất 0,076 s. Đại lượng đáng so ở đây vì thế là chi phí một vòng lặp, $\mathcal{O}(nd^2 + d^3)$ so với $\mathcal{O}(nd)$ của gradient descent, chứ không phải số vòng lặp.



$$w_1 := w_0 - (\nabla^2 f)^{-1} \nabla f(w_0) := w^*$$

Với hàm bậc hai, một vòng lặp là đủ và bước Newton chính là nghiệm đúng, nên toàn bộ lời giải mất 0.076 s. Đại lượng đáng so ở đây vì thế là chi phí một vòng lặp, $\mathcal{O}(nd^2 + d^3)$ so với $\mathcal{O}(nd)$ của gradient descent, chứ không phải số vòng lặp.



12:15–13:20. Gradient descent chỉ biết hướng dốc, Newton biết cả độ cong nên không mò từng bước mà nhảy thẳng tới đáy của xấp xỉ bậc hai tại chỗ đang đứng. Ở đây f vốn là hàm bậc hai, xấp xỉ bậc hai của nó chính là nó, nên xuất phát từ $w_0 = 0$ thì một vòng lặp ra đúng w^* . Nói cách khác, bước Newton chính là nghiệm đúng.

Vì vậy đại lượng đáng so ở đây là chi phí một vòng lặp chứ không phải số vòng lặp: $\mathcal{O}(nd^2 + d^3)$ so với $\mathcal{O}(nd)$ của gradient descent. Toàn bộ lời giải mất 0,076 s.

Ba biến thể Newton trên biểu đồ gần như trùng nhau, và cần giải thích đúng lý do: backtracking

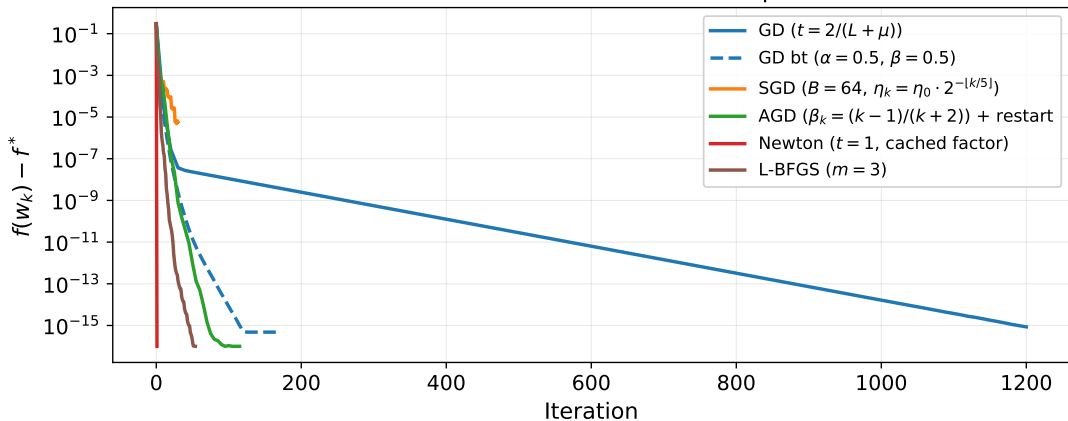
chấp nhận ngay $t = 1$ nên chỉ tốn một lần đánh giá hàm, còn phân rã lại hay dùng lại thì vô nghĩa khi chỉ có một vòng lặp. Hai cặp này chỉ tách nhau trên bài toán phi tuyến.

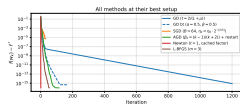
Nếu bị hỏi vì sao dùng Cholesky chứ không phải `np.linalg.inv`: nghịch đảo tường minh kém ổn định về số học và tốn nhiều phép tính hơn phân rã rồi giải thế.

Nếu bị hỏi vì sao Newton-CG chậm hơn Newton 25 lần: vì $n \gg d$ ở bài này. Một lần Cholesky tốn $\mathcal{O}(d^3) \approx 2 \cdot 10^7$, còn mỗi tích Hessian nhân vector tốn $\mathcal{O}(nd) \approx 4.5 \cdot 10^7$, mà CG cần nhiều tích như vậy. Quan hệ chỉ đảo khi d lớn tới mức không lập nổi ma trận $d \times d$.

So sánh theo số vòng lặp

All methods at their best setup





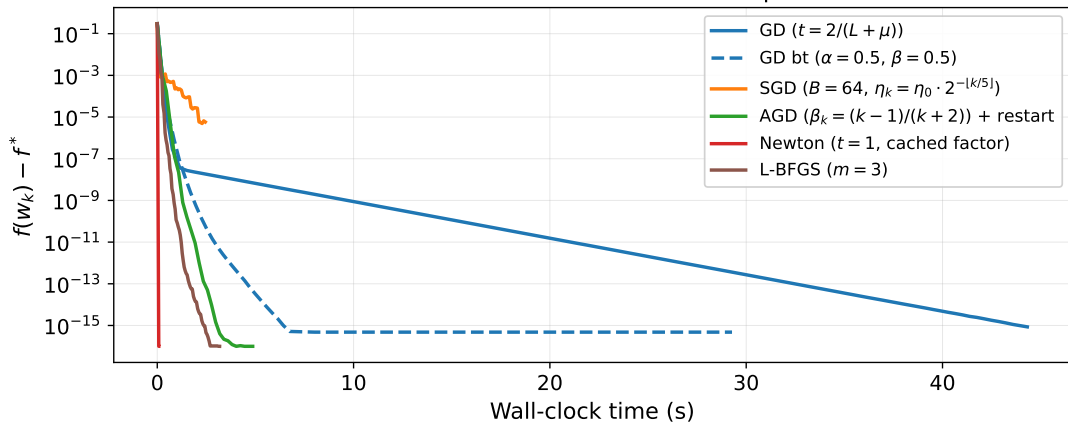
13:20–14:20. Biểu đồ gom cả bảy thuật toán trên cùng một hàm mục tiêu, trục tung là $f(w_k) - f^*$ trên thang logarit, trục hoành là số vòng lặp.

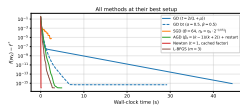
Đọc biểu đồ này thì Newton nằm tách hẳn ở góc trái, đúng một vòng lặp là xuống đáy, L-BFGS mất 8 vòng, còn ba phương pháp bậc một bám sát nhau suốt phần đầu rồi mới tách ở đuôi. SGD là đường

nằm ngang phía trên, chưa chạm 10^{-6} sau 30 lượt duyệt.

Chỗ cần nói trước khi sang slide sau: trục vòng lặp đối xử với mọi vòng lặp như nhau, mà một vòng Newton tốn $\mathcal{O}(nd^2 + d^3)$ còn một vòng gradient descent chỉ tốn $\mathcal{O}(nd)$. Nên biểu đồ này chưa trả lời được câu hỏi thuật toán nào chạy nhanh, và đó chính là lý do phải có biểu đồ thứ hai.

All methods at their best setup





14:20–15:25. Cùng bảy thuật toán, đối trục hoành sang giây, và thứ hạng đổi.

Hai chỗ đáng chỉ tay vào. Thứ nhất, SGD tụt xuống rất nhanh ở đoạn đầu vì mỗi lần cập nhật chỉ đọc 64 dòng, nhưng rồi nằm ngang trong khi các đường khác vẫn đi xuống. Thứ hai, AGD và GD trùng nhau tới quãng 10^{-6} rồi tách hẳn: đi tới giới hạn số học, AGD mất 4,86 s còn GD mất 44,3 s, chậm hơn 9 lần.

Cơ chế của chỗ tách ấy là ở giai đoạn đầu, các hướng có độ cong lớn chỉ phôi sai số, mà mọi phương pháp bậc một đều cắt các hướng ấy nhanh như nhau. Chỉ tới phần đuôi, khi μ mới là thứ chi phối, lợi thế của gia tốc mới hiện ra.

Đề bài bắt vẽ cả hai biểu đồ chính vì chỗ này, và nếu chỉ được nhớ một câu trong cả bài thì nên nhớ câu vừa rồi.

Bảng tổng hợp

Thuật toán	Cấu hình tốt nhất	Vòng lặp	Thời gian (s)	Tổng (s)
Newton	$t = 1$, Cholesky	1	0,076	0,076
L-BFGS	$m = 3$	8	0,448	3,18
GD	$t = 2/(L + \mu)$	20	0,773	44,32
GD	backtracking $\alpha = \beta = 0.5$	15	0,852	29,28
AGD	β_k tăng dần, khởi động lại	20	0,879	4,86
SGD	$B = 64$, giảm bậc thang	–	–	2,45

- ▶ Cột *Vòng lặp* và *Thời gian* đo tới ngưỡng 10^{-6} , cột *Tổng* đo tới giới hạn số học.
- ▶ Ở 10^{-6} , ba phương pháp bậc một chênh nhau dưới 14% vì giai đoạn đầu do các hướng có độ cong lớn chi phối; ở độ chính xác cao, nơi μ mới chi phối, AGD nhanh hơn GD 9 lần.
- ▶ L-BFGS là một trong ba thuật toán cài đặt thêm ngoài bốn thuật toán bắt buộc, trình bày ở phụ lục.



Thuật toán	Câu lệnh tối thiểu	Vòng lặp	Thời gian (s)	Tổng (s)
Newton	$\epsilon = 1$, Cholesky	1	0,076	0,076
L-BFGS	$m = 1$	8	0,448	3,18
GD	$\epsilon = 2/(1 + \mu)$	20	0,773	14,32
GD	backtracking $\alpha = \beta = 0,5$	15	0,852	20,28
AGD	β_i tăng dần, khởi động lại	20	0,879	4,86
SGD	$\beta_i = 1/i$, giảm bậc thang	--	--	2,45

- Cột *Vòng lặp* và *Thời gian* do tôi ngưỡng 10^{-6} , cột *Tổng* do tôi giới hạn số học.
- Ở 10^{-6} , ba phương pháp bậc một chênh nhau dưới 14% vì giai đoạn đầu do các hướng có độ trung tâm chỉ phụ thuộc vào độ chính xác của, nó phụ thuộc vào μ , AGD nhanh hơn GD 9 lần.
- L-BFGS là một trong ba thuật toán cài đặt thêm ngoài bốn thuật toán bậc hai, trình bày ở phụ lục.

15:25–16:25. Bảng này cho ba kết luận, và nhóm em xin nói đúng ba câu đó.

Một, Newton nhanh nhất trên bài này, 0,076 s cho toàn bộ lời giải, ít hơn 10 lần thời gian GD cần chỉ để đạt 10^{-6} . Nhưng phải nói kèm điều kiện: hàm mục tiêu là bậc hai nên bước Newton chính là nghiệm đóng, và khoảng cách này không nói gì về bài toán phi tuyến.

Hai, ở độ chính xác vừa phải thì chọn phương pháp bậc một nào cũng như nhau, cả ba đạt 10^{-6} trong

0,77 tới 0,88 giây, chênh dưới 14%.

Ba, ở độ chính xác cao thì thứ hạng đảo lộn, AGD nhanh hơn GD 9 lần. Cột *Vòng lặp* và *Thời gian* đo tới 10^{-6} , còn cột *Tổng* đo tới giới hạn số học, và chênh lệch giữa hai cột cuối chính là chỗ đáng bàn nhất của cả bảng.

Nếu bị hỏi về L-BFGS: chuyển tới slide phụ lục về ba thuật toán cài đặt thêm. Nói gọn thì nó không lưu Hessian, chỉ giữ m cặp gần nhất, chi phí $\mathcal{O}(md)$ mỗi vòng, nên đứng giữa hai nhóm về cả chi phí lẫn tốc độ.

Bộ ước lượng	sai số $f - f^*$	thời gian (s)	RMSE test
Ridge(solver='cholesky')	0	0,112	0,206 01
Ridge(solver='lsqr')	$2,35 \times 10^{-9}$	0,621	0,206 01
Ridge(solver='sag')	$1,46 \times 10^{-8}$	3,955	0,206 01
SGDRegressor (mặc định)	$1,16 \times 10^{-3}$	1,018	0,212 02
LinearRegression	$3,48 \times 10^{-5}$	1,116	0,205 79

- ▶ Nhóm quy hàm mục tiêu của thư viện về cùng một dạng trước khi so sánh, $\alpha = \lambda n$ cho Ridge và $\alpha = \lambda$ cho SGDRegressor, và kiểm chứng cả hai phép quy đổi bằng số.
- ▶ Solver trực tiếp giải thẳng chính hệ mà một bước Newton giải, nên với bài toán có nghiệm đóng và d vài trăm thì khó có phương pháp lặp nào sánh được.
- ▶ SGDRegressor mặc định dừng cách f^* một khoảng $1,16 \times 10^{-3}$, gấp 200 lần cấu hình SGD tự cài đặt, vì quy tắc chọn bước mặc định không tính theo L_B của bài toán.



Bộ ước lượng	sai số $f - f^*$	thời gian (s)	RMSE test
<code>Ridge(alpha=0.0001)</code>	0	0,112	0,20000
<code>Ridge(alpha='lap')</code>	$2,35 \times 10^{-6}$	0,002	0,20000
<code>Ridge(alpha='ag')</code>	$1,06 \times 10^{-6}$	3,005	0,20000
<code>SGDRegressor (mặc định)</code>	$1,16 \times 10^{-3}$	1,016	0,21262
<code>LinearRegression</code>	$0,00 \times 10^{-6}$	1,116	0,20579

- Nhóm quy hàm mục tiêu của thư viện về cùng một dạng trước khi so sánh, $\alpha = \lambda n$ cho Ridge và $\alpha = \lambda$ cho SGDRegressor, và kiểm chứng cả hai phép quy đổi bằng số.
- Solver trực tiếp giải thẳng chỉnh hệ mà một bước Newton giải, nên với bài toán có nghiệm đúng và ở cái tiền thì khó có phương pháp lặp nào sánh được.
- SGDRegressor mặc định dừng cách f^* một khoảng $1,16 \times 10^{-3}$, gấp 200 lần cấu hình SGD tự cài đặt, vì quy tắc chọn bước mặc định không tính theo L_B của bài toán.

16:25–17:55. Trước khi so sánh phải quy hàm mục tiêu về cùng một dạng, vì mỗi thư viện chuẩn hóa hệ số một kiểu. Ridge cực tiểu hóa $\|Xw - y\|^2 + \alpha \|w\|^2$, không có $1/2n$ và không có $\lambda/2$, nên $\alpha = \lambda n = 1600$. Nhóm em kiểm chứng bằng số chứ không tin suy luận trên giấy: nghiệm Ridge cách nghiệm đúng $1,06 \times 10^{-14}$, và chênh lệch hàm mục tiêu bằng đúng 0.

Hai dòng đáng nói trong bảng. Solver trực tiếp đạt f^* với sai số cỡ sai số máy trong 0,112 s, vì nó giải thẳng chính cái hệ mà một bước Newton giải. Còn

SGDRegressor ở tham số mặc định dừng cách f^* tận $1,16 \times 10^{-3}$, gấp 200 lần cấu hình SGD nhóm em tự cài, vì quy tắc chọn bước mặc định của nó không tính theo L_B nên rơi đúng vào cái lân cận đã nói ở phần SGD. Cần nói rõ giới hạn: nhận xét này áp cho bộ ước lượng đó ở tham số mặc định, không áp cho scikit-learn nói chung.

Câu chốt phần này: mục đích tự cài đặt không phải để chạy nhanh hơn thư viện, mà để biết tham số mặc định của thư viện có hợp bài toán của mình không, và biết mỗi thuật toán nhanh hay chậm ở giai đoạn nào, vì sao.

- ▶ **Độ dài bước:** $2/L$ là ranh giới thật, chênh 10% đủ để đổi giữa hội tụ và phân kỳ, nên trong vùng ổn định cứ chọn càng gần $2/L$ càng tốt.
- ▶ **Backtracking:** β quyết định chi phí còn α gần như không; ghép với phương pháp tăng tốc thì phải lấy $\alpha = 1/2$ hoặc $t_0 = 1/L$, và momentum phải tính theo bước thực tế.
- ▶ **SGD:** độ dài bước phải theo kích thước lô, vì $L_{\max}/L = 1159$ trên bộ dữ liệu này.
- ▶ **Bậc hai:** Newton chính là nghiệm đóng nên nhanh hơn hẳn, nhưng kết luận đó gắn với dạng bậc hai và không chuyển sang bài toán phi tuyến.
- ▶ **Lý thuyết và thực nghiệm:** cận chặt ở phần đuôi và lỏng ở giai đoạn đầu, vì phổ trị riêng của bài toán tập trung chứ không trải đều.



- **Dữ liệu bước:** $2/L$ là ranh giới thực, chính 10% đủ để đổi giữa hội tụ và phân kỳ, nên trong vòng ổn định có chọn càng gần $2/L$ càng tốt.
- **Backtracking:** β quyết định chỉ phí còn α gần như không; ghép với phương pháp tăng tốc thì phải lấy $\alpha = 1/2$ hoặc $\alpha = 1/L$, và momentum phải tính theo bước thực là.
- **SGD:** dữ liệu bước phải theo hình thức là, ví $k_{max}/L = 1529$ tiến bộ đó liên này
- **Bậc hai:** Newton chính là nghiệm đúng nên nhanh hơn hẳn, nhưng kết luận đó gần với dạng bậc hai và không chuyển sang bài toán phi tuyến.
- **Lý thuyết và thực nghiệm:** cần chốt ở phần dưới và tăng ở giai đoạn đầu, vì phổ trị năng của bài toán tập trung chủ không trái dấu.

17:55–19:10. Năm dòng này là năm câu nếu nhóm em chỉ được nói năm câu, nên đọc chậm và không thêm gì.

Dòng đầu và dòng ba nói về độ dài bước, và cả hai quy về một mốc duy nhất là $2/L$. Dòng bốn là chỗ nhóm em muốn nhấn nhất, vì nó nối cả bài lại: cùng một bảng số cho hai kết luận trái ngược nhau tùy vào độ chính xác mà ta cần, và không có biểu đồ thứ hai theo thời gian thì không thấy được.

Dòng cuối là giới hạn của cả bài, nên nói ra chứ đừng để hội đồng phải hỏi. Kết luận Newton nhanh nhất gần chặt với dạng bậc hai có nghiệm đúng, và

trên bài toán phi tuyến thì nó không còn đúng.

Nếu còn thời gian thì thêm một câu về vai trò kép của λ : nó vừa chống quá khớp vừa cải thiện điều kiện của bài toán, mà hai vai trò ấy không cùng chiều. Chi tiết nằm ở slide phụ lục.

Em xin hết. Nhóm em xin lắng nghe câu hỏi của thầy cô.

Về nhíp nói: các mốc thời gian trên đầu mỗi trang tính theo 150 tiếng một phút và cộng lại vào khoảng 19 phút, tức còn dư 50 giây. Chỗ dư đó dành cho phần hỏi đáp xen giữa chứ không phải để nói thêm, nên trang nào bị chậm quá 30 giây thì nên bỏ bớt câu ở trang sau thay vì nói nhanh lên.

- ▶ Chọn λ trên một đường cong CV phẳng.
- ▶ Chuẩn hóa dữ liệu, cách tạo biến tương tác và số điều kiện.
- ▶ SGD: ảnh hưởng của kích thước lô khi mỗi lô có bước riêng.
- ▶ Đối chiếu với tốc độ lý thuyết.
- ▶ Quy đổi hàm mục tiêu của thư viện, và so sánh theo thời gian.
- ▶ Ảnh hưởng của λ tới số điều kiện và tới RMSE.
- ▶ Ba thuật toán cài đặt thêm.
- ▶ Kiểm thử tính đúng đắn của cài đặt.



- ▶ Chọn λ trên một đường cong CV phẳng.
- ▶ Chuẩn hóa dữ liệu, cách tạo biến tương tác và xử điều kiện.
- ▶ SGD: ảnh hưởng của kích thước dữ liệu mỗi lần cập nhật bước.
- ▶ Dữ liệu với tốc độ lý thuyết.
- ▶ Quy đổi hàm mục tiêu của thư viện, và so sánh theo thời gian.
- ▶ Ảnh hưởng của λ tới xử điều kiện và tới RMSE.
- ▶ Thuật toán cải thiện.
- ▶ Kiểm thử tính đúng đắn của cài đặt.

Slide điều hướng, không chiếu trong phần nói chính. Khi hội đồng hỏi thì mở slide này ra, đọc đúng một dòng khớp với câu hỏi rồi nhảy thẳng tới slide đó, đừng lật tuần tự.

Ánh xạ nhanh từ câu hỏi sang slide: hỏi về chọn λ thì vào dòng một; hỏi vì sao phải chuẩn hóa thì dòng

hai; hỏi SGD chạy chậm hay đường nằm ngang thì dòng ba; hỏi vì sao thực nghiệm không khớp cận lý thuyết thì dòng bốn; hỏi về so sánh với thư viện thì dòng năm; hỏi λ ảnh hưởng gì tới tốc độ thì dòng sáu; hỏi về L-BFGS, Heavy ball, Newton-CG hay Adam thì dòng bảy; hỏi làm sao biết cài đúng thì dòng tám.

Chọn λ trên một đường cong CV phẳng

- ▶ Đường cong CV 5 fold cho sai số giống nhau tới bốn chữ số thập phân với mọi λ từ 10^{-6} đến 10^{-3} , vì mức hiệu chỉnh nhỏ hơn nhiều của dữ liệu thì không đổi được sai số dự báo.
- ▶ Cùng khoảng đó, $\kappa = L/\lambda$ chênh nhau 1000 lần, nên λ vẫn phải chọn cẩn thận bằng một tiêu chí ngoài cross-validation.
- ▶ Trên đường cong phẳng, điểm cực tiểu chỉ phản ánh nhiều của phép chia fold, nên nhóm dùng quy tắc một sai số chuẩn: chọn λ lớn nhất còn nằm trong một sai số chuẩn của giá trị tốt nhất.
- ▶ Kết quả $\lambda = 0,01$, $\kappa = 268,3$, đôi lấy RMSE test 0,206 01 so với 0,205 79, kém 0,1%.



- Đường cong CV 5 fold cho sai số giống nhau tới bốn chữ số thập phân với mọi λ từ 10^{-6} đến 10^{-3} , vì mức biến thiên nhỏ hơn nhiều của dữ liệu thử không đủ được sai số dự báo.
- Công khoảng đó, $\kappa \approx 1/5$ chính nhau 1000 lần, nên λ vẫn phải chọn cẩn thận bằng một tiêu chí ngoài cross-validation.
- Trên đường cong phẳng, điểm cực tiểu chỉ phản ánh nhiễu của phép chia fold, nên nhóm dùng quy tắc một sai số chuẩn: chọn λ lớn nhất của năm trong một sai số chuẩn của giá trị tối thiểu.
- Kết quả $\lambda = 0,01$, $\kappa = 208,3$, độ tỷ RMSE test 0,206/01 so với 0,205/73, kém 0,1%.

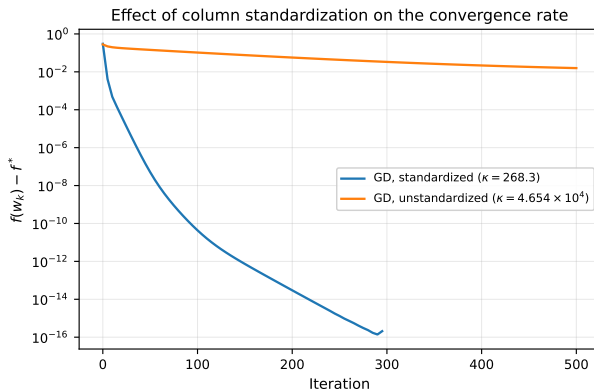
Dùng khi bị hỏi vì sao không lấy điểm cực tiểu của cross-validation. Trả lời: trên một đường cong phẳng tới bốn chữ số thập phân, điểm cực tiểu chỉ phản ánh nhiễu của phép chia fold chứ không phản ánh bài toán. Nhóm em đổi 0,1% RMSE lấy κ nhỏ đi 100 lần.

Nói kèm điều kiện đảo chiều, vì hội đồng hay hỏi

tiếp: đánh đổi này có lợi vì đường cong phẳng và vì nhóm em giải bằng phương pháp lặp. Nếu đường cong có cực tiểu rõ thì mất 0,1% là mất thật, còn nếu chỉ dùng phương pháp bậc hai thì κ không chỉ phôi số vòng lặp nên khoản được cũng biến mất.

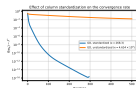
Slide này cũng là chỗ phân biệt hai bài toán: chọn λ là bài toán học máy, làm một lần lúc chuẩn bị; cực tiểu hóa f với λ đã cho mới là nội dung môn học.

Chuẩn hóa dữ liệu và số điều kiện



$\kappa = 268,3$ sau khi chuẩn hóa so với $\kappa = 46\,540$ khi giữ thang đo không đồng nhất, và hệ số co rút $1 - \mathcal{O}(1/\kappa)$ nhỏ đi đúng theo tỉ lệ 173 lần ấy.





$\kappa \approx 2000.3$ sau khi chuẩn hóa so với $\kappa \approx 40540$ khi giữ thang đo không đổi, và hệ số có rất ít $1 - O(1/\kappa)$ nhỏ đi đúng theo tỉ lệ 173 lần ấy.

Dùng khi bị hỏi chuẩn hóa dữ liệu có liên quan gì tới môn tối ưu hóa. Cách nói cho dễ hình dung: cột giá gốc có giá trị cỡ 10^5 , cột số inch màn hình cỡ 6, nên chưa chuẩn hóa thì cái bát bị kéo dài theo hướng giá gốc thành một cái máng dài hàng chục nghìn lần. Chuẩn hóa là nắn cái máng đó lại thành cái bát.

Con số để chốt: cùng 500 vòng lặp, bản chuẩn hóa chạm giới hạn số học sau khoảng 290 vòng, bản giữ thang gốc mới xuống 10^{-2} . Chi phí mỗi vòng không đổi, đều khoảng 42 ms, nên biểu đồ theo thời gian chỉ là biểu đồ này vẽ lại trên thang khác.

Nói kèm giới hạn: lợi ích ấy chỉ dành cho nhóm bậc một, vì Newton vẫn hội tụ sau đúng một vòng lặp bất kể κ bằng bao nhiêu.

Ảnh hưởng của cách tạo biến tương tác tới κ

Tích của hai cột định lượng có thể lấy trên thang gốc hoặc trên cột đã chuẩn hóa.

	thang gốc	cột đã chuẩn hóa
L	17,14	2,68
số trị riêng $< 10^{-6}$	48	7

Không gian cột sinh ra vẫn như cũ, chỉ cách biểu diễn thay đổi, nhưng một cột tích trên thang gốc có phương sai lớn hơn các cột còn lại nhiều bậc và kéo riêng trị riêng lớn nhất lên, nên số điều kiện chênh nhau 6,4 lần.



Tích của hai cột định lượng có thể lấy trên thang gốc hoặc trên cột đã chuẩn hóa.

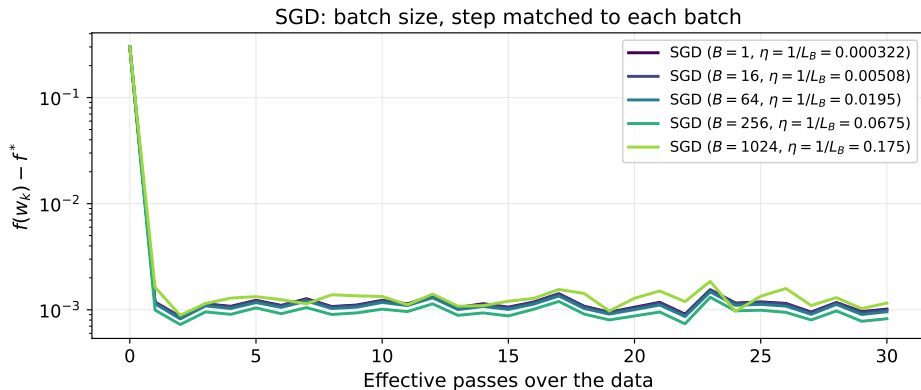
	thang gốc	cột đã chuẩn hóa
$\bar{z}$	17.14	2.68
giá trị riêng < 10^{-6}	48	7

Không gian cột sinh ra vẫn như cũ, chỉ cách biểu diễn thay đổi, nhưng một cột tích trên thang gốc có phương sai lớn hơn các cột của lại nhiều bậc và làm nhiễu tín hiệu nhất tín, nên sẽ điều kiện chính nhau 6,4 lần.

Dùng khi bị hỏi tạo biến tương tác thế nào. Chênh lệch trong bảng đến từ một phép nhân: `original_price` cỡ 10^5 nhân với `screen_size_inches` cỡ 6 thì ra một cột cỡ 10^6 , và một mình nó kéo trị riêng lớn nhất lên.

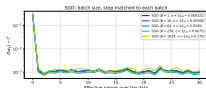
Điểm cần nhấn: không gian cột sinh ra vẫn y hệt nhau nên mô hình không mạnh lên hay yếu đi, chỉ cách biểu diễn thay đổi. Nhân trên cột đã chuẩn hóa đưa số trị riêng dưới 10^{-6} từ 48 về 7, đúng bằng mức của khối cột gốc.

SGD: ảnh hưởng của kích thước lô



Khi mỗi lô có bước riêng, cả năm cấu hình về cùng mức 10^{-3} sau 30 lượt duyệt, nhưng thời gian chênh 14 lần, 27,4s với $B = 1$ so với 1,9s với $B = 1024$. Chênh lệch nằm ở chi phí cố định của mỗi lần cập nhật trong Python chứ không ở số phép tính, do $B = 1$ thực hiện 160 000 lần cập nhật mỗi lượt duyệt.





Khi mỗi lô số bước tăng, cả năm cấu hình về cùng mức 10^{-3} sau 30 lượt duyệt, nhưng thời gian chạy 14 lần, 27.4s với $B=1$ so với 1.2s với $B=1024$. Chính việc nằm ở chi phí cá định của mỗi lần cập nhật trong Python chứ không ở số phép tính, do $B=1$ thực hiện 160000 lần cập nhật mỗi lượt duyệt.

Dùng khi bị hỏi vì sao chọn $B=64$. Điểm cần làm rõ trước: biểu đồ này chạy với mỗi lô một bước riêng $\eta_0 = 1/L_B$, khác hẳn thí nghiệm đối chứng dùng chung một bước ở phần chính. Khi bước đã chỉnh theo lô thì cả năm cấu hình về cùng mức 10^{-3} , tức kích thước lô không còn quyết định độ chính xác, chỉ còn quyết định thời gian.

Nói rõ chệch lệch 14 lần đến từ đâu, vì đây là chỗ dễ

hiểu nhầm: nó là chi phí điều phối của Python chứ không phải số phép tính, do $B=1$ chạy 160 000 lần cập nhật mỗi lượt duyệt mà mỗi lần chỉ xử lý một dòng.

Nếu bị hỏi tiếp con số 14 lần có phổ quát không thì trả lời là không, và ghi rõ đây mới là dự đoán chứ nhóm em chưa đo: cài bằng ngôn ngữ biên dịch, hoặc d lớn hơn nhiều để phần số học lần át, thì tỉ lệ sẽ co lại.

	Quan sát	Cận lý thuyết
GD, $t = 2/(L + \mu)$	0,985 20	0,985 20
AGD, β_k tăng dần + restart	0,654 56	0,938 95

- ▶ Cận của GD chặt tới năm chữ số thập phân ở phần đuôi, vì ở đó đúng thành phần ứng với trị riêng nhỏ nhất chi phối sai số.
- ▶ Nhưng nó nói rất ít về giai đoạn đầu, khi GD đạt 10^{-6} sau 20 vòng lặp còn ước lượng lý thuyết là $\frac{1}{2}\kappa \ln(10^6) \approx 1850$.
- ▶ Phổ trị riêng của bài toán tập trung chứ không trải đều giữa μ và L như trường hợp xấu nhất mà cận nhắm tới, và cùng lý do đó khiến momentum tăng dần thắng momentum hằng.



	Quan sát	Cận lý thuyết
GD, $\tau = 2/(k + \mu)$	0,98520	0,98520
AGD, β_k tăng dần + restart	0,65456	0,93805

- Cận của GD nhạy và sâu chữ số thập phân ở phần dưới, vì ở đó đúng thành phần ứng với trị riêng nhỏ nhất chi phối sai số.
- Nhưng nó nói rất ít về giai đoạn đầu, khi GD đạt 10^{-6} sau 20 vòng lặp của ước lượng lý thuyết là $\log_{10}(UP) \approx 155$.
- Phổ trị riêng của bài toán tập trung chủ yếu không trải đều giữa μ và L như trường hợp cầu nhất mà cận nhảm tả, và cũng lý do đó khiến momentum tăng dần thắng momentum hằng.

Dùng khi bị hỏi thực nghiệm có khớp lý thuyết không. Nhóm em đo hệ số co rút bằng cách khớp một đường thẳng vào phần tuyến tính ở đuôi của đồ thị $\log_{10}(f(w_k) - f^*)$ theo k .

Hai dòng của bảng phải đọc theo hai kiểu khác nhau. GD trùng cận tới năm chữ số thập phân, và không phải trùng ngẫu nhiên: ở đuôi, đúng thành phần ứng với trị riêng nhỏ nhất chi phối sai số, mà tốc độ tắt

của thành phần đó chính là $(\kappa - 1)/(\kappa + 1)$. AGD thì tốt hơn hẳn cận của nó, 0,65 so với 0,94, và đó không phải mâu thuẫn vì cận là cận trên cho trường hợp xấu nhất.

Chênh lệch lớn nhất nằm ở giai đoạn đầu: lý thuyết ước lượng khoảng 1850 vòng lặp để đạt 10^{-6} , thực tế chỉ 20 vòng, nhỏ hơn gần hai bậc. Kết luận thực dụng: đừng dùng cận lý thuyết để lập ngân sách vòng lặp cho bài toán này.

- ▶ Ridge cực tiểu hóa $\|Xw - y\|_2^2 + \alpha\|w\|_2^2$, nên $\alpha = \lambda n$.
- ▶ SGDRegressor cực tiểu hóa đúng dạng (1), nên $\alpha = \lambda$.

$$f(w) = \frac{1}{2n}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2 \quad (1)$$

Nhóm kiểm chứng cả hai phép quy đổi bằng số chứ không bằng suy luận trên giấy: nghiệm Ridge cách nghiệm đóng $1,06 \times 10^{-14}$, và chênh lệch hàm mục tiêu bằng đúng 0 trong số học dấu phẩy động.



► Ridge cực tiểu hóa $\|Xw - y\|^2 + \alpha\|w\|^2$, nên $\alpha = \lambda n$.

► SGDRegressor cực tiểu hóa đúng dạng (1), nên $\alpha = \lambda$.

$$f(w) = \frac{1}{2n} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2 \quad (1)$$

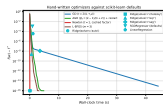
Nhóm kiểm chứng cả hai phép quy đổi bằng số chỉ không bằng nay luận trên giấy; nghiệm Ridge cách nghiệm đúng 1.06×10^{-14} , và chính lịch hàm mục tiêu bằng đúng 0 trong số học dấu phẩy động.

Dùng khi bị hỏi so sánh với thư viện có công bằng không. Trả lời bằng con số: $\alpha = \lambda n = 0,01 \times 160\,000 = 1600$ cho Ridge, còn SGDRegressor cực tiểu hóa đúng dạng của nhóm em nên $\alpha = \lambda$.

Nhấn vào cách kiểm chứng chứ không chỉ vào công thức: nhóm em viết phép kiểm tra này thành một khẳng định trong notebook thay vì một dòng in ra để đọc bằng mắt, vì mọi so sánh phía sau sẽ vô nghĩa nếu nó thất bại.

Kết quả so với thư viện, theo thời gian





Dùng khi bị hỏi cài đặt của nhóm so với thư viện thì thế nào. Trả lời thẳng: trên bài này thư viện thắng, Ridge với solver trực tiếp mất 0,112s và không phương pháp lặp nào ở quy mô $d = 280$ sánh được, vì nó giải thẳng chính cái hệ mà một bước Newton giải.

Nhưng nói tiếp về sau, vì đó mới là ý của bài tập: mục đích tự cài đặt không phải để chạy nhanh hơn thư viện. Nó cho hai thứ mà một lần gọi hàm không cho, là biết tham số mặc định của thư viện có hợp bài toán của mình không, và biết mỗi thuật toán nhanh hay chậm ở giai đoạn nào, vì sao. Phương pháp lặp lấy lại chỗ đứng khi bài toán lớn tới mức không giải trực tiếp được.

λ	κ	Vòng lặp đạt 10^{-6}	Thời gian (s)	RMSE test
0,001	2674	40	1,756	0,205 80
0,01	268,3	40	1,771	0,206 01
0,1	27,73	30	1,233	0,219 16

- ▶ Vì $\mu = \lambda$, tăng λ mười lần thì giảm κ đúng mười lần.
- ▶ Nhưng κ giảm 96 lần chỉ đổi được số vòng lặp từ 40 xuống 30, trong khi RMSE xấu đi 6,5%, vì giai đoạn đầu do các hướng có độ cong lớn chi phối chứ không do κ .
- ▶ Hiệu chỉnh vì thế có hai vai trò không cùng chiều, thống kê và tối ưu hóa, và đánh đổi giữa chúng chỉ có lợi khi cần độ chính xác cao.

λ	κ	Vòng lặp đạt 10^{-6}	Thời gian (s)	RMSE test
0.001	207.1	40	1.756	0.20510
0.01	209.3	40	1.771	0.20610
0.1	27.73	30	1.233	0.21916

- ▶ Vì $\mu = \lambda$, tăng λ mười lần thì giảm κ đúng mười lần.
- ▶ Nhưng κ giảm 96 lần chỉ đủ để số vòng lặp từ 40 xuống 30, trong khi RMSE xấu đi 6,5%, vì giai đoạn đầu do các hướng có độ cong lớn chi phối chứ không do κ .
- ▶ Hiệu chỉnh vì thế có hai vai trò không cùng chiều, thông kê và tối ưu hóa, và đánh đổi giữa chúng chỉ có lợi khi cần độ chính xác cao.

Dùng khi bị hỏi λ dính gì tới tốc độ hội tụ. Slide này nói chủ đề Ridge với nội dung môn học, nên nếu còn thời gian ở phần chính thì cũng đáng chiếu. Vì $\mu = \lambda$ trên bài toán này, tăng λ mười lần là giảm κ đúng mười lần, tức hệ số hiệu chỉnh có luôn một vai trò tối ưu hóa bên cạnh vai trò thống kê quen thuộc.

Nhưng con số trong bảng nói rằng hai vai trò ấy lệch nhau rất xa. Từ $\lambda = 0.001$ lên 0.1, κ giảm 96 lần

mà số vòng lặp chỉ giảm từ 40 xuống 30, trong khi RMSE xấu đi 6,5%. Số vòng lặp ít nhạy cảm như vậy vì cùng lý do đã nói ở phần so sánh tổng hợp, rằng giai đoạn đầu do các hướng có độ cong lớn chi phối chứ không do κ .

Chốt: tăng λ để cải thiện điều kiện là đánh đổi không có lợi nếu chỉ cần độ chính xác vừa phải, và nó chỉ đáng giá khi cần độ chính xác cao, nơi phần đuôi mới tốn kém.

- ▶ **Heavy ball.** Cùng dạng momentum như Nesterov nhưng ngoại suy sau khi lấy gradient:
 $w_{k+1} = w_k - t \nabla f(w_k) + \beta(w_k - w_{k-1})$, với $t = 4/(\sqrt{L} + \sqrt{\mu})^2$ và
 $\beta = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2$.
- ▶ **Newton-CG.** Giải hệ Newton bằng gradient liên hợp. Mỗi bước CG chỉ cần một tích Hessian nhân vector, $\frac{1}{n} X^\top (Xv) + \lambda v$, chi phí $\mathcal{O}(nd)$, nên không phải lập ma trận $d \times d$. Ở đây chậm hơn Newton 25 lần vì $d = 280$ còn nhỏ.
- ▶ **L-BFGS.** Xấp xỉ nghịch đảo Hessian từ m cặp (s_k, y_k) gần nhất qua đệ quy hai vòng, chi phí $\mathcal{O}(md)$ mỗi vòng lặp. Nằm giữa nhóm bậc một và bậc hai về cả chi phí lẫn tốc độ, đạt 10^{-6} sau 8 vòng lặp và 0,448 s.

Adam cũng được cài đặt nhưng không đưa vào so sánh chính, vì trên hàm bậc hai lồi mạnh có nghiệm đóng, cơ chế chia theo căn bậc hai của moment bậc hai không mang lại lợi thế nào.



- **Heavy ball.** Công dụng momentum như Newton nhưng ngoại suy sau khi lấy gradient:

$$w_{k+1} = w_k - \gamma \nabla f(w_k) + \beta(w_k - w_{k-1}), \text{ với } \beta = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2} \text{ và } \beta = \left(\frac{\sqrt{L} - 1}{\sqrt{L} + 1} \right)^2.$$

- **Newton-CG.** Giải hệ Newton bằng gradient liên hợp. Mỗi bước CG chỉ cần một tích Hessian nhân vector, $\frac{1}{2} \nabla^2 f(x_k) = \lambda x_k$, chi phí $\mathcal{O}(nd)$, nên không phải lặp ma trận $d \times d$. Ở đây chọn hơn Newton 25 lần vì $d = 280$ còn nhỏ.

- **L-BFGS.** Xấp xỉ nghịch đảo Hessian từ m cặp (m, y) gần nhau qua độ quay hai vòng, chi phí $\mathcal{O}(md)$ mỗi vòng lặp. Liên giữa nhóm bậc một và bậc hai về cả chi phí lẫn tốc độ, đạt 10^{-6} sau 8 vòng lặp và 0,168 s.

Adam cũng được cài đặt nhưng không đưa vào so sánh chính, vì trên hàm bậc hai thì mạnh mẽ nghiệm đúng, cơ chế chia theo căn bậc hai của moment bậc hai không mang lại lợi thế nào.

Dùng khi bị hỏi ngoài bốn thuật toán bắt buộc còn gì. Ba thuật toán này nằm ngoài yêu cầu của đề, nhóm em cài thêm để lấp khoảng giữa nhóm bậc một và nhóm bậc hai.

Nếu bị hỏi riêng về L-BFGS thì có hai quan sát đáng kể. Chọn m không quan trọng ở giai đoạn đầu, cả bốn cấu hình đạt 10^{-6} sau đúng 8 vòng lặp, vì trong 8 vòng đó số cặp tích lũy được còn ít hơn cả $m = 3$.

Tới cuối thì m mới có tác dụng, từ 54 vòng với $m = 3$ xuống 37 vòng với $m = 20$.

Nhưng tổng thời gian không giảm đều theo m : $m = 10$ tốn 3,95 s trong khi $m = 3$ chỉ tốn 3,18 s dù cần nhiều hơn 14 vòng lặp. Chênh lệch đến từ line search, vốn tiêu 227 lần đánh giá hàm ở $m = 10$ so với 105 lần ở $m = 5$. Với $d = 280$ thì chi phí $\mathcal{O}(md)$ nhỏ hơn hẳn một lần đánh giá f , nên line search mới là phần chi phối chứ không phải m .

46 phép thử tự động chạy trước khi khảo sát tham số. Bốn phép thử đối chiếu trực tiếp với công thức giải tích:

- ▶ gradient giải tích so với sai phân trung tâm tại $\varepsilon = 10^{-6}$, sai số tương đối dưới 10^{-6} ;
- ▶ Hessian so với sai phân của gradient, cùng ngưỡng;
- ▶ nghiệm đóng là điểm dừng, $\|\nabla f(w^*)\| < 10^{-12}$;
- ▶ mười câu hình thuật toán cùng hội tụ về một nghiệm trên bài toán tổng hợp $n = 100$, $d = 5$.

Ba phép thử về mặt phương pháp luận: gradient descent phải phân kỳ khi $t > 2/L$; SGD với bước hằng phải dừng cách f^* một khoảng dương; ghi lịch sử dày hay thưa phải cho cùng số lần đánh giá hàm mục tiêu.



48 phép thử tự động chạy trước khi khảo sát tham số. Bản phép thử đối chiếu trực tiếp với công thức giải tích:

- gradient giải tích so với sai phân trung tâm tại $x = 10^{-10}$, sai số tương đối dưới 10^{-15} ;
 - Hessian so với sai phân của gradient, cũng tương tự;
 - nguyên hàm là hàm đơn: $|\nabla^2 f(w)| \leq 10^{-10}$;
 - mọi câu lệnh thuật toán cũng hội tụ về một nghiệm trên bài toán tổng hợp $n = 100$, $d = 5$.
- Ba phép thử về một phương pháp luận: gradient descent phải phân kỳ khi $\epsilon > 2/L$; SGD với bước tăng phải cũng cách f^* một khoảng dương; ghi lịch sử dự hay thua phải cho cùng số lần đánh giá hàm mục tiêu.

Dùng khi bị hỏi làm sao biết cài đặt đúng. Ý cần truyền đạt: bài toán có sẵn nghiệm đóng và biểu thức giải tích, nên nhóm em không phải tin vào việc đọc lại mã nguồn mà đối chiếu được với mốc tính ra bằng tay.

Ba phép thử về phương pháp luận đáng nói riêng, vì chúng bắt thuật toán phải có hành vi *xấu* đúng như lý thuyết dự đoán. Nếu GD không phân kỳ khi $t > 2/L$, hoặc SGD bước hằng lại hội tụ về đúng

f^* , thì có gì đó sai chứ không phải cài đặt tốt.

Kể hai lỗi thật đã bắt được, vì cả hai đều để chương trình chạy trót lọt rồi trả về một con số trông hợp lý. Một, quy tắc nhận diện cột số có đơn vị biến `model_36` thành số 36, làm mất cột định tính nhiều mức nhất. Hai, momentum của AGD tính theo $1/L$ nhưng ghép với bước do line search chọn, làm hàm mục tiêu bùng lên $1,47 \times 10^4$. Không có mốc giải tích thì cả hai còn nguyên trong kết quả.